Universite Abdelmalek Essaadi

Ecole Nationale des Sciences Appliquees - Al Hoceima

COMPTE RENDU

TP N 3 - Programmation Orientee Objet en Java

Etudiant	JARDINI ABDELAZIZ
Filiere	ID1 - Semestre 2
Annee universitaire	2025-2026
Encadrant	Pr. Mohamed CHERRADI
Email soumission	mcherradi.ensah@gmail.com
Objet du mail	TP3

Exercice 1 : Classe Point

Description

La classe Point represente un point dans un espace 2D avec ses coordonnees x et y. Elle offre des constructeurs (avec et sans parametres), une methode d'affichage, de deplacement et de calcul de distance a l'origine et entre deux points.

Code source - EX1.java

```
import java.lang.Math;

class Point {
    double x, y, distance;

    public Point() {
        this.x = 1;
        this.y = 2;
    }

    public Point(double X, double Y) {
        this.x = X;
        this.y = Y;
    }

    public void afficher() {
        System.out.println("L'abscisse est " + x + " et L'ordonnee est " + y);
    }

    public void deplacement(double dx, double dy) {
        x = x + dx;
        y = y + dy;
    }

    public double calcul() {
        return Math.sqrt(Math.pow(x, 2) + Math.pow(y, 2));
    }

    public double diff(double x1, double y1, double x2, double y2) {
        return Math.sqrt(Math.pow(x1 - x2, 2) + Math.pow(y1 - y2, 2));
    }
}

public class EX1 {
    public static void main(String[] args) {
        Point p1 = new Point();
        Point p2 = new Point(3, 4);
        Point p3 = new Point(4, 5);
        p1.afficher(); p2.afficher(); p3.afficher();
        p1.deplacement(1, 1); p2.deplacement(1, 1); p3.deplacement(1, 1);
        System.out.println("apres deplacement");
        p1.afficher(); p2.afficher(); p3.afficher();
        System.out.println("le point p1: " + p1.calcul());
        System.out.println("le point p2: " + p2.calcul());
        System.out.println("le point p3: " + p3.calcul());
        System.out.println("la distance est :" + p1.diff(p1.x,p1.y,p2.x,p2.y));
    }
}
```

Resultat d'execution :

```
L'abscisse est 1.0 et L'ordonnee est 2.0
L'abscisse est 3.0 et L'ordonnee est 4.0
L'abscisse est 4.0 et L'ordonnee est 5.0
apres deplacement
L'abscisse est 2.0 et L'ordonnee est 3.0
L'abscisse est 4.0 et L'ordonnee est 5.0
```

```
L'abscisse est 5.0 et L'ordonnee est 6.0
```

Exercice 2 : Classe Rectangle

Description

La classe Rectangle modelise un rectangle defini par sa largeur et sa hauteur. Elle calcule la surface et le perimetre.

Code source - EX2.java

```
class Rectangle {
    double largeur, hauteur;

    public Rectangle(double l, double h) {
        this.largeur = l;
        this.hauteur = h;
    }

    public double surface() {
        return largeur * hauteur;
    }

    public double perimetre() {
        return (largeur + hauteur) * 2;
    }
}

public class EX2 {
    public static void main(String[] args) {
        Rectangle r = new Rectangle(11, 2);
        System.out.println("la surface est : " + r.surface());
        System.out.println("le perimetre est : " + r.perimetre());
    }
}
```

Resultat d'execution :

```
la surface est : 22.0
le perimetre est : 26.0
```

Exercice 3 : Classe Etudiant

Description

La classe Etudiant gere les informations d'un etudiant (CNE sous forme de tableau et note). Elle possede une methode estAdmis() (note >= 12) et toString() pour l'affichage.

Code source - EX3.java

```
class Etudiant {
    int[] cne;
    double note;

    public Etudiant(int[] cne, double note) {
        this.cne = cne;
        this.note = note;
    }

    public boolean estAdmis() {
        return note >= 12;
    }

    public String toString() {
        String cneStr = "";
        for (int i = 0; i < cne.length; i++) {
            cneStr += cne[i];
        }
        return "CNE: " + cneStr + " Note: " + note + " Admis: " + estAdmis();
    }
}

public class EX3 {
    public static void main(String[] args) {
        int[] cne = {1, 2, 3, 4, 5};
        Etudiant e = new Etudiant(cne, 13.5);
        System.out.println(e.toString());
    }
}
```

Resultat d'execution :

```
CNE: 12345 Note: 13.5 Admis: true
```

Exercice 4 : Classe NombreComplexe

Description

La classe complexe represente un nombre complexe avec ses parties reelle et imaginaire. Elle offre les operations d'addition et de multiplication, ainsi qu'une methode d'affichage.

Code source - EX4.java

```
class complexe {
    int imaginaire;
    int reel;

    public complexe(int a, int b) {
        this.reel = a;
        this.imaginaire = b;
    }

    public complexe addition(complexe c) {
        return new complexe(this.reel + c.reel, this.imaginaire + c.imaginaire);
    }

    public complexe multiplication(complexe c) {
        int r = this.reel * c.reel - this.imaginaire * c.imaginaire;
        int i = this.reel * c.imaginaire + this.imaginaire * c.reel;
        return new complexe(r, i);
    }

    public void affichage() {
        System.out.println(this.reel + " + i" + this.imaginaire);
    }
}

public class EX4 {
    public static void main(String[] args) {
        complexe c1 = new complexe(1, 2);
        complexe c2 = new complexe(3, 4);
        complexe somme   = c1.addition(c2);
        complexe produit = c1.multiplication(c2);
        somme.affichage();
        produit.affichage();
    }
}
```

Resultat d'execution :

```
4 + i6
-5 + i10
```

Exercice 5 : Classe CompteBancaire

Description

La classe comptebancaire modelise un compte bancaire avec un numero et un solde. Elle permet de deposer, retirer, transferer des fonds et d'afficher le solde.

Code source - EX5.java

```
class comptebancaire {
    int numero;
    double solde;

    public comptebancaire(int n, double s) {
        this.numero = n;
        this.solde = s;
    }

    public void deposer(double add_value) { solde += add_value; }
    public void retirer(double mins_value) { solde -= mins_value; }

    public void transferer(comptebancaire c, double montant) {
        this.solde -= montant;
        c.solde += montant;
    }

    public void afficher_solde() {
        System.out.println("Solde: " + solde);
    }
}

public class EX5 {
    public static void main(String[] args) {
        comptebancaire c1 = new comptebancaire(1234, 1000);
        comptebancaire c2 = new comptebancaire(4321, 500);
        c1.transferer(c2, 200);
        c1.afficher_solde();
        c2.afficher_solde();
    }
}
```

Resultat d'execution :

```
Solde: 800.0
Solde: 700.0
```

Exercice 6 : Classe Voiture

Description

La classe voiture modelise une voiture avec sa marque et sa vitesse. La methode accelerer() limite la vitesse a 240 km/h et freiner() empeche la vitesse de passer en dessous de 0.

Code source - EX6.java

```
class voiture {
    String marque;
    int vitesse;

    public voiture(String m, int v) {
        this.marque = m;
        this.vitesse = v;
    }

    void accelerer(int value) {
        if ((vitesse + value) > 240) {
            System.out.println("vous avez depasser la vitesse maximale");
            vitesse = 240;
        } else {
            vitesse += value;
            System.out.println("votre vitesse actuelle est : " + vitesse);
        }
    }

    void freiner(int value) {
        if ((vitesse - value) < 0) { vitesse = 0; }
        else { vitesse -= value; }
        System.out.println("votre vitesse actuelle est : " + vitesse);
    }
}

public class EX6 {
    public static void main(String[] args) {
        voiture v1 = new voiture("BMW", 100);
        v1.accelerer(50);
        v1.freiner(30);
        v1.freiner(150);
    }
}
```

Resultat d'execution :

```
votre vitesse actuelle est : 150
votre vitesse actuelle est : 120
votre vitesse actuelle est : 0
```

Exercice 7 : Classe Produit & GestionProduit

Description

La classe produit modelise un article avec id, nom, description, prix et quantite. La classe gestionProduit utilise une ArrayList pour gerer une collection de produits avec des operations CRUD completes.

Code source - EX7.java

```
class produit {
    int id; String nom; String description; double prix; int quantite;

    public produit(int id, String nom, String description, double prix, int
quantite) {
        this.id = id;  this.nom = nom;  this.description = description;
        this.prix = prix;  this.quantite = quantite;
    }
}

class gestionProduit {
    ArrayList<produit> liste = new ArrayList<>();

    public void ajouterProd(produit p) { liste.add(p); }

    public void modifierProd(int id, String nom, String description,
                             double prix, int quantite) {
        for (produit p : liste) {
            if (p.id == id) {
                p.nom = nom; p.description = description;
                p.prix = prix; p.quantite = quantite;
            }
        }
    }

    public void supprimerProd(int id) {
        liste.removeIf(p -> p.id == id);
    }

    public produit getProduitByID(int id) {
        for (produit p : liste) { if (p.id == id) { return p; } }
        return null;
    }
}

public class EX7 {
    public static void main(String[] args) {
        gestionProduit gp = new gestionProduit();
        produit p1 = new produit(1, "Or",   "rare",  1000, 5);
        produit p2 = new produit(2, "Zinc", "metal",  200, 10);
        gp.ajouterProd(p1);  gp.ajouterProd(p2);
        gp.modifierProd(1, "Or", "pur", 1500, 3);
        produit p = gp.getProduitByID(1);
        if (p != null) { System.out.println(p.nom + " " + p.prix); }
        gp.supprimerProd(2);
    }
}
```

Resultat d'execution :

```
Or 1500.0
```

Exercice 7 - Bonus : Deviner un Nombre Aleatoire

Description

Le programme genere un nombre aleatoire entre 1 et 10. L'utilisateur doit le deviner en un minimum de tentatives. Un score base sur le nombre de tentatives est affiche a la fin.

Code source - EX7_bonus.java

```
import java.util.Scanner;

class nb_secret {
    int secret;
    int tentatives = 0;

    public nb_secret() {
        this.secret = (int)(Math.random() * 11);
    }

    public void deviner() {
        Scanner sc = new Scanner(System.in);
        int x;
        do {
            System.out.print("Entrer un nombre : ");
            x = sc.nextInt();
            tentatives++;
            if (x < secret)      { System.out.println("Plus grand"); }
            else if (x > secret) { System.out.println("Plus petit"); }
            else                 { System.out.println("Bravo !"); }
        } while (x != secret);
        System.out.println("Score (tentatives) : " + tentatives);
    }
}

public class EX7_bonus {
    public static void main(String[] args) {
        nb_secret jeu = new nb_secret();
        jeu.deviner();
    }
}
```

Resultat d'execution :

```
Entrer un nombre : 2
Bravo !
Score (tentatives) : 1
```

Conclusion

Ce TP a permis de mettre en pratique les concepts fondamentaux de la Programmation Orientee Objet en Java, notamment :

- La declaration de classes avec des attributs et des constructeurs (avec et sans parametres).
- L'encapsulation des donnees et l'utilisation du mot-cle this.
- La creation et la manipulation d'objets (instanciation, appel de methodes).
- L'utilisation de structures de controle (if/else, boucles for, do-while).
- La gestion de collections dynamiques avec ArrayList (Exercice 7).
- La manipulation de nombres aleatoires et d'entrees utilisateur via Scanner (Bonus).

L'ensemble des exercices a ete compile et execute avec succes sous VS Code avec l'extension Java.